Customer Churn Prediction in Telecom Industry: A Machine Learning Based Customer Retention System

Anantula Sai Nikhila Sravani
Roll No. 23K91A0512
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

Avula Anudeep
Roll No. 23K91A0518
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

B Sandeep
Roll No. 24K915A0503
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

Daragani Vinod Kumar
Roll No. 23K91A0558
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

B. Srikanth
Assistant Professor
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

***Abstract*---**Telecom operators lose revenue when customers leave, and a churn score without a reason gives a retention team nothing to act on. This paper presents ChurnIQ, an end-to-end customer retention system: a scikit-learn pipeline that imputes, scales and encodes customer records, handles class imbalance with class weights or SMOTE, trains and compares Logistic Regression, Decision Tree and Random Forest classifiers, groups customers into editable risk bands, explains each score through its largest drivers and turns them into rule-based retention actions, behind a FastAPI service and a React interface. No real operator data were available, so all experiments use 1,000 synthetic customers with the public Telco schema and 26.4 % churn. On a 200-row test split Logistic Regression reaches F1 0.486 and ROC-AUC 0.722; cross-validation and repeated splits show spreads of 0.03 to 0.10, so the leading models are not separable. Probabilities are not calibrated, SHAP was not exercised and the retention rules were not evaluated on campaign data. The results are not comparable with the real-data results of the base paper.

***Index Terms*---**churn prediction, customer retention, explainable AI, imbalanced data, machine learning, random forest, telecom analytics.

I. INTRODUCTION

Customer churn, the loss of subscribers to a competitor, is the main way a telecom operator's customer base erodes [1], and retaining a customer is generally treated as cheaper than acquiring a new one [2], [3]. Operators already hold early signals of churn in their records: tenure, contract type, monthly charges, the services taken and the way the bill is paid. Machine learning turns these records into a churn probability, and comparisons of standard classifiers on this task are well established [3], [4], [5].

Two practical problems remain. A bare score gives a retention team nothing to act on: it does not say why a customer is at risk or what to offer, which is why recent work adds explanation methods such as SHAP [6] and perturbation-based explainers [1]. And because churners are a minority (26.5 % of the public Telco data [7]), a classifier can reach a high accuracy while missing most of them, so the metric and the handling of imbalance decide the outcome [8]. Most studies stop at a table of metrics; few deliver an application that scores every customer, shows the drivers of each score and maps them to actions.

The problem addressed is therefore: given a table of customer records, estimate for every customer the probability of leaving, group customers into risk bands, show which attributes drive each prediction and turn the result into a concrete retention action, in an application that an analyst can run without writing code and with metrics that are not fooled by class imbalance.

This paper describes ChurnIQ, an applied extension of the motivation of the base paper [1], and makes four contributions: (1) a tested pipeline from a customer CSV to a ranked, banded customer list, with class weights or SMOTE [9] for imbalance and model selection on F1; (2) per-customer explanation and a transparent rule-based retention engine; (3) a FastAPI service and a React interface over one shared core, with batch scoring and a drift check; and (4) a measured baseline that reports cross-validation spreads, calibration and decision-threshold behaviour beside the headline metrics, on clearly declared synthetic data. Section II reviews related work, Section III describes the method, Section IV reports the results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11, no GPU.

B. Software Requirements

Python 3.11, scikit-learn 1.9.1 [10], pandas 3.0.6, imbalanced-learn, FastAPI 0.143.3 [11], React 19 with Vite 8 for the interface, and pytest for testing.

II. LITERATURE SURVEY

Base paper. Oprea and Bara [1] ask which customer attributes matter most for retention and how to find them quickly. They run four perturbation-based explainers on a trained churn model (random noise perturbation, feature

ablation, input marginalization and adversarial perturbation), combine them into one importance score and keep rows with a positive recall drop. On the Telco data (7,043 customers, 21 columns, 26.5 % churn; 70/30 split, 2,113 test rows) they compare Logistic Regression, SVC, Random Forest, Balanced Random Forest, CatBoost, XGBoost and LightGBM with recall as the primary objective; recall reaches 0.9077 for CatBoost at a precision of 0.4719. Their unified score agrees with SHAP (Spearman 0.8125, Kendall 0.7126, Jaccard 0.6885) and is much faster (18.5 s against 289.4 s for SHAP and 35.1 s for LIME in their setting). They find month-to-month contracts, monthly charges above about $90 and tenure below about 20 months to carry the highest risk, propose CRM integration as a decision-support layer, and state that their conclusions depend on recall as the objective and on a structured, moderate-dimensional data set.

Churn prediction. Comparisons of classifiers on churn data are the norm [3] and have been extended to big-data platforms [2]. Sikri et al. [4] add a ratio-based balancing step for imbalanced churn data and find ensembles better than single models, XGBoost at a 75:25 ratio being best on their measures. El Attar and El-Hajj [5] compare eight models on the Telco data (80/20 split) with SHAP feature analysis and report XGBoost at accuracy 0.84 and ROC-AUC 0.932 against 0.78 and 0.864 for Logistic Regression; Rasool et al. [12] report accuracy 94.75 %, recall 73.20 % and F1 80.23 % for a stacking ensemble with SHAP on a 3,333-customer data set. Explainable churn models have also been built with monotonic decision rules [13] and with leverage-point analysis [14]. These figures are quoted as reported; the studies differ in data, split and balancing and cannot be ranked against one another or against this work.

Explaining predictions. SHAP [6] attributes a prediction to features with Shapley values and has a fast exact form for trees [15]; LIME [16] fits a local surrogate model. Both are perturbation-based, are costly or less consistent in the comparison of [1], and can be fooled by adversarial models [17], so an explanation describes a model and not the customer's reasons.

TABLE I
REPRESENTATIVE WORK COMPARED

Work	Approach	Data	Reported result	Limitation
[1]	4 perturbation explainers vs SHAP, LIME; 7 classifiers	Telco, 7,043 rows	Recall up to 0.9077; Spearman 0.8125 with SHAP	One data set; recall-only objective
[4]	Ratio-based balancing, ensembles	Imbalanced telecom churn	XGBoost best at 75:25 ratio	Abstract-level reading; no explanation described
[5]	8 models, SHAP	Telco, 80/20	XGBoost acc. 0.84, AUC 0.932	Single data set; SHAP not causal
[12]	Stacking, SHAP	3,333 customers	Acc. 94.75 %, recall 73.20 %	No temporal model
[6]	Shapley attributions	Method paper	Unifies attribution methods	289.4 s in [1]
[16]	Local surrogate model	Method paper	Local explanations of any classifier	Lower consistency than SHAP in [1]
[17]	Adversarial attack on explainers	Method paper	Explainers can be fooled	Read explanations with caution

Imbalance and classifier families. Churners are a minority, and the usual remedies are cost-sensitive learning and resampling; He and Garcia [8] survey them, and SMOTE [9] creates synthetic minority examples by interpolation. The classifier families in the churn literature are linear models, decision trees, random forests [18] and gradient boosting [19], the last being the family behind the XGBoost, LightGBM and CatBoost models of the base paper. This work uses the first three as its required set because their behaviour can be read directly, and adds two scikit-learn boosting models as options.

Positioning. The base paper [1] answers how to explain a churn model quickly and recommends integration into a CRM as a decision-support layer; the studies of classifier performance [4], [5], [12] answer how accurate a model can be on one data set. This work takes the explanation motivation of the first and the comparison practice of the second and builds the missing layer between them: an application that scores, bands, explains and recommends, and whose evaluation states its own uncertainty.

Gap. The surveyed work compares classifiers or explainers on churn data. Few papers pair accurate prediction with a working application that scores every customer, explains each score and links the drivers to actions that can be audited, and the reported numbers depend so strongly on split, balancing and data set that a single-split metric can mislead. This paper addresses the application and reports the spreads.

III. METHODOLOGY

Pipeline. A customer CSV (or the generated sample) is validated and its target and identifier columns are detected; the data are split in a stratified 80/20 ratio, a preprocessor is fitted on the training split only, imbalance is handled, models are trained and compared, the best is saved, and every customer is scored and placed in a risk band with drivers and actions. Fig. 1 shows the architecture and Fig. 2 the main flow.

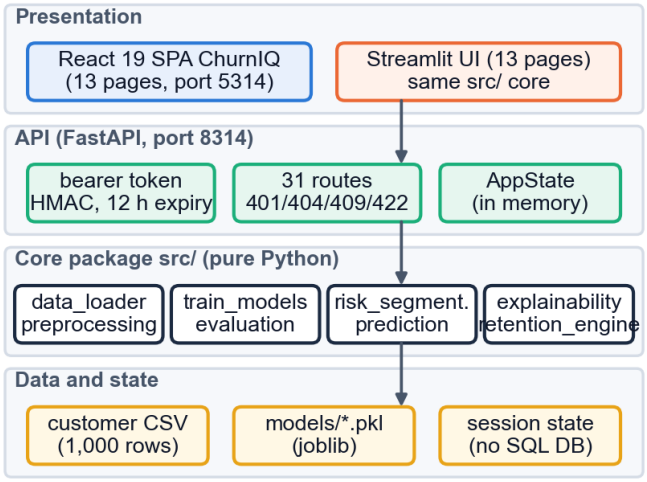


Fig. 1. Architecture of ChurnIQ: two interfaces, a FastAPI service, the pure-Python core src/ and the data and state.

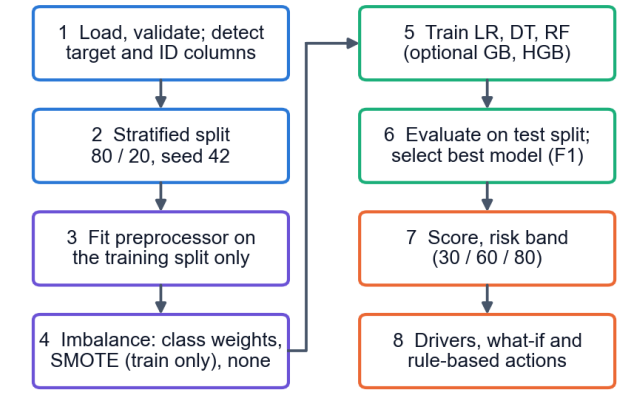


Fig. 2. Main flow from a customer table to explained, banded customers.

Data. The public Telco data [7] were not used because no real operator data were available to the team; the generator in the project (src/sample_data.py, seed 42) produces 1,000 customers with the same 21 columns (4 numeric and 15 categorical features). Churn is drawn from a latent risk score that grows with month-to-month contracts, fibre-optic service, electronic-check payment, missing support add-ons and higher charges and falls with tenure (55 % of customers are on month-to-month contracts, 44 % have fibre optic and 34 % pay by electronic check), plus Gaussian noise; the churn rate is 26.4 %. Models trained on it can recover these effects, which checks the pipeline but says nothing about real customers.

Preprocessing and imbalance. Numeric fields get median imputation and standardisation; categorical fields get most-frequent imputation and one-hot encoding, giving 45 inputs. The split gives 800 training and 200 test rows (53 churners). Class weights set w_c = n / (2 n_c), i.e. 1.896 for churners and 0.679 for retained customers; SMOTE [9] interpolates between minority neighbours on the transformed training split only; a third option applies no handling.

Models. Logistic Regression models P(churn | x) = 1 / (1 + exp(-(b0 + w.x))) and, with standardised inputs, w_j x_j is the contribution of feature j. A Decision Tree (depth 8, at least 10 customers per leaf) splits on Gini impurity. A Random Forest [18] averages 300 trees (depth 12, leaf 5). Gradient Boosting [19] (200 trees, depth 3, rate 0.08) and Histogram Gradient Boosting are optional. All come from scikit-learn [10] with fixed settings and no hyper-parameter search. The model is chosen on F1 and not on accuracy, because with 26 % churners a model that never predicts churn is 74 % accurate. Metrics are accuracy, precision, recall, F1 and ROC-AUC on the held-out split.

Evaluation protocol. With TP, FP, TN and FN the counts of the confusion matrix and churn as the positive class, precision = TP / (TP + FP), recall = TP / (TP + FN) and F1 = 2 P R / (P + R); ROC-AUC is the probability that a random churner is scored above a random retained customer. Because one split of 200 rows is noisy, the three required models are also evaluated by stratified 5-fold cross-validation (200 held-out rows per fold) and on five random 80/20 splits (seeds 0 to 4), and the mean and standard deviation are reported. Imbalance handling is compared by repeating the single-split experiment with class weights, SMOTE and no handling. The decision threshold and the calibration of the scores are examined on the test predictions of the saved Logistic Regression model.

Risk bands and actions. The probability (predict_proba x 100) is placed in Low (below 30), Medium, High (60 to 80) or Critical (80 and above) with thresholds that the user can edit and the server validates. Nine of ten IF/THEN retention rules (for example a month-to-month contract, no tech support, monthly charges of 80 or more, electronic check, tenure of 6 months or less) fire only at a probability of 60 or more; the tenth fires in the Critical band, and each suggestion carries its reason.

Explanation. Global importance uses coefficients or tree importances and permutation importance (at most 300 test rows, 5 repeats). The local explanation multiplies each feature's weight by its standardised value and lists the five largest positive and negative contributions; an optional SHAP path exists but SHAP was not installed, so it was not exercised. A what-if page re-scores a customer after attribute changes; a batch page scores a CSV (predicted churn at 50 % or more) and a drift check compares new data with the training data.

Design decisions. All logic lives in the pure-Python core src/, which imports nothing from either interface, so the Streamlit pages and the FastAPI routes cannot disagree; the preprocessor is fitted on the training split only and SMOTE touches only the training split, so no test information leaks; settings and uploads are validated on the server and refused with 401, 404, 409 or 422 and a plain message; and nothing leaves the machine: no external service, no language model and no database server. The cost of this simplicity is a single-user, in-memory design.

Batch scoring and monitoring. A batch file must contain the feature columns the active model was trained on;

otherwise the service answers 422 and names the missing columns. The output has the customer identifier, the probability, predicted churn (probability of 50 or more), the risk band and, optionally, the three largest risk factors; a 25-row template can be downloaded. The drift check compares a new file with the training data: missing columns, the percentage of missing values (flagged above 10 %), categories not seen in training, the shift of each numeric mean and the churn rate of the new file. It is a screening tool and computes no statistical drift test.

Implementation. The FastAPI layer has 31 route handlers, bearer-token authentication (HMAC-SHA256, 12 h), server-side validation and an in-memory state; only the active model is saved (joblib) and restored. The React interface has 13 pages. 44 unit tests and a 63-check end-to-end API test pass.

IV. RESULTS

Test conditions. All numbers come from scripts that call the project's own functions: 1,000 synthetic customers, stratified 80/20 split with seed 42, class weights, a 200-row test set with 53 churners, scikit-learn 1.9.1 and pandas 3.0.6. Table II gives the five models. Logistic Regression is the best on F1, ROC-AUC and recall and is the saved model; Gradient Boosting has the highest accuracy (0.745) and precision (0.536) but a recall of only 0.283, and the Decision Tree is the weakest of the three required models.

TABLE II
TEST-SPLIT METRICS (80/20, SEED 42, CLASS WEIGHTS)

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Regression	0.630	0.385	0.660	0.486	0.722
Decision Tree	0.615	0.338	0.472	0.394	0.608
Random Forest	0.665	0.405	0.566	0.472	0.711
Gradient Boosting	0.745	0.536	0.283	0.370	0.697
Hist Gradient Boosting	0.710	0.400	0.189	0.256	0.685

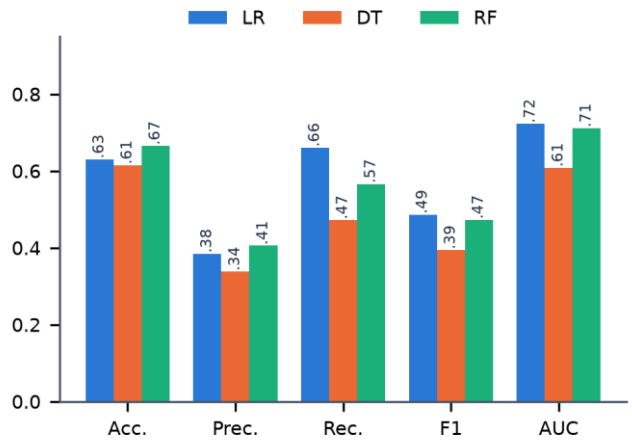


Fig. 3. Accuracy, precision, recall, F1 and ROC-AUC of the three required models on the test split.

Portfolio and speed. The saved model places the 1,000 customers in the four bands as 253, 481, 213 and 53 (Figs. 4 and 5). Training all five models takes 2.96 s, scoring one customer a median 19.07 ms and all 1,000 customers 35.4 ms; API responses over 127.0.0.1 have a median of 65 to 94 ms for what-if, Customer 360 and the dashboard.

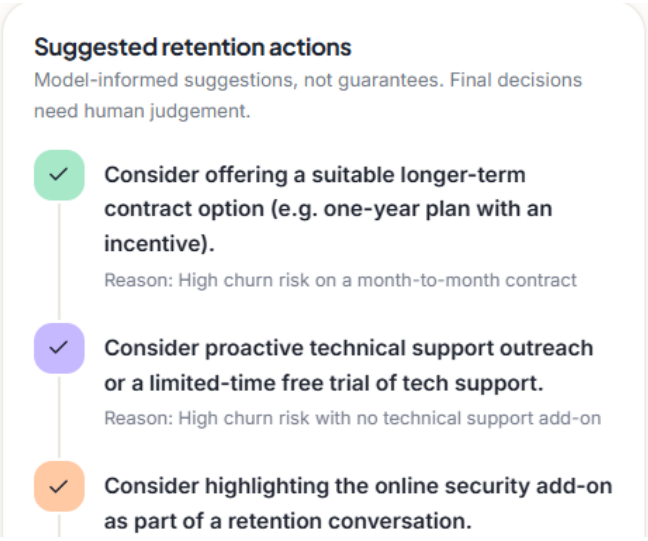


Fig. 4. Output screen: retention actions of Customer 360 for the top-ranked customer, each with the reason that triggered it.

Confusion matrices. At the default 0.50 cut-off Logistic Regression finds 35 of the 53 churners and raises 56 false alarms; Random Forest finds 30 with 44 false alarms, the Decision Tree 25 with 49, Gradient Boosting 15 with 13 and Histogram Gradient Boosting 10 with 15. The choice between them is a trade-off between wasted offers and lost customers.

Risk bands and a worked example. Because class weights raise every score (mean 45.89 % against 26.4 % real churn), 266 customers (26.6 %) fall in the High or Critical bands; the bands rank customers and are not chances of leaving. Fig. 4 shows the retention actions that Customer 360 gives the highest-ranked customer, scored 92.1 % (Critical, tenure 4 months, recorded outcome churned): the month-to-month contract, the missing tech-support add-on and the missing online-security add-on each trigger a rule, and each suggestion is shown with its reason so that an agent can accept or reject it.

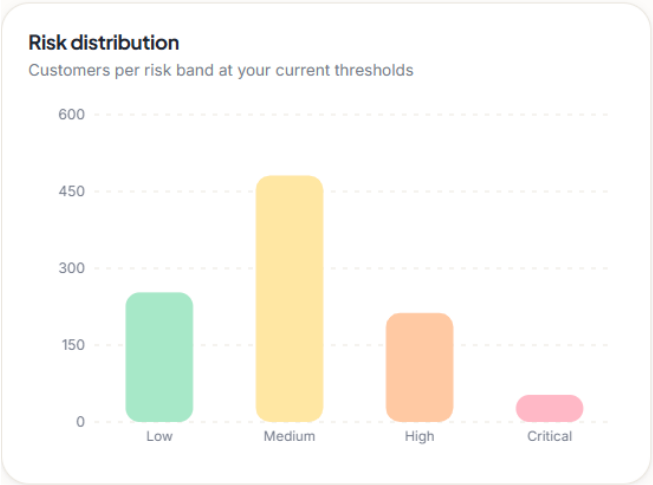


Fig. 5. Output screen: customers per risk band at the default 30/60/80 thresholds.

Stability (Fig. 6). A 200-row test split is noisy. In 5-fold cross-validation the F1 of Logistic Regression is 0.483 ± 0.055 and of Random Forest 0.463 ± 0.051 (Decision Tree 0.431 ± 0.028); ROC-AUC is 0.695 ± 0.046 and 0.697 ± 0.042. Over five random splits the F1 is 0.507 ± 0.044 and 0.496 ± 0.033. Logistic Regression and Random Forest are within one standard deviation of each other, so the data do not support calling either one better; ROC-AUC of about 0.70 to 0.72 is a moderate ranking ability, not a strong one.

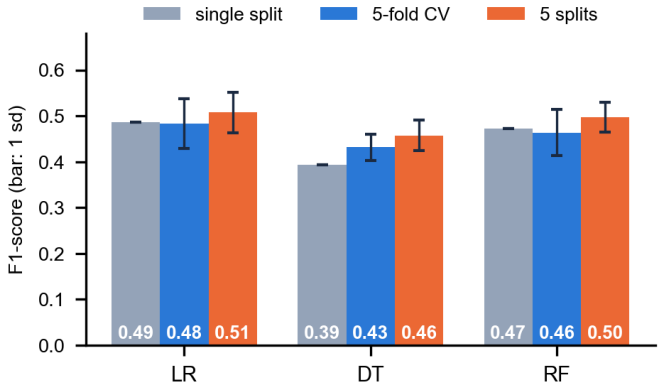


Fig. 6. F1-score of the three required models on the single split, in 5-fold cross-validation and on five random splits (bars: one standard deviation).

Imbalance. Without any handling, accuracy looks fine (0.745 for Logistic Regression, 0.755 for Random Forest) but recall collapses to 0.189 and 0.094. SMOTE gives the best single-split F1 for Logistic Regression (0.526) but was tried on one split only and is not claimed to beat class weights.

Drivers. The largest Logistic Regression coefficients are contract type (month-to-month 0.825, two-year 0.791), monthly charges (0.558) and tenure (0.400); permutation importance is led by monthly charges (0.042) and month-to-month contracts (0.028). These are the effects built into the generator, so they confirm the pipeline, and they match the drivers reported for real data in the base paper [1] only in the sense that both contain them.

Threshold, calibration and targeting (Fig. 7). At the 50 % cut-off of the batch table precision is 0.385, recall 0.660 and F1 0.486; F1 stays between 0.503 and 0.509 from 0.40 to 0.60, while at 0.30 recall is 0.962 but precision 0.325. The probabilities are not calibrated: the mean prediction is 0.461 against an observed rate of 0.265 and the Brier score is 0.213, no better than the 0.195 of a constant predictor, so scores are rankings. Contacting the top 20 % of customers (40) reaches 21 of the 53 churners (39.6 %, lift 1.98); the top 10 % (20 customers) contain 9 (precision 0.45) and the top 50 % contain 41 (77.4 %). A lift of 1.5 to 2 is a moderate gain, and with 20 to 100 contacts the counts are small.

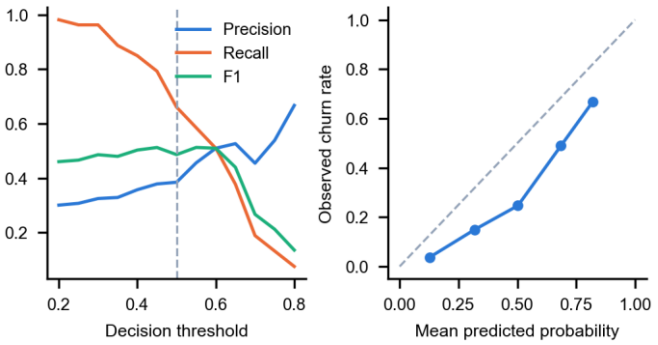


Fig. 7. Left: precision, recall and F1 of Logistic Regression against the decision threshold. Right: reliability of the predicted probabilities.

Comparison with the base paper. The data differ: the base paper [1] uses the real 7,043-row Telco data with a 2,113-row test set and boosted models tuned for recall, whereas this work uses 1,000 synthetic customers, a 200-row test set and untuned scikit-learn models. Its accuracy of 0.699 to 0.793 and recall of 0.411 to 0.908 and this work's accuracy of 0.615 to 0.745 and recall of 0.189 to 0.660 are therefore not comparable, and no claim is made to match or beat them. The base paper's unified perturbation explainer and LIME were not implemented.

Testing. The 44 unit tests cover loading and target normalisation, the stratified split, SMOTE, the band boundaries (0, 29.99, 30, 59.9, 60, 79.9, 80, 100), batch validation, the retention rules, model persistence and the token; the 63-check end-to-end test also exercises the error paths (401, 404, 409, 422). Writing them exposed a real defect: the data loader tested the pandas dtype against object in three places, which is false for the string dtype of pandas 3, so blank TotalCharges strings were not converted and labels such as Churned and Retained were mishandled; it was fixed and both pandas versions now pass. Calls through the hostname localhost took 2.04 s longer than through 127.0.0.1 on this Windows machine because of IPv6 resolution, so latencies were measured on 127.0.0.1.

Threats to validity. The churn label is generated from the same features the model sees, so the relationships are cleaner than in real data and the scores would probably be lower on a real operator's customers. The test set holds 53 churners, so differences of a few points are not reliable; the spreads above, not the single-split figures, should guide any conclusion. Tree-ensemble results depend on the library version. The explanations come from coefficients and permutation importance and were not compared with SHAP.

Failure cases and limits. Logistic Regression flags 56 of the 147 retained customers at 0.50 and misses 18 of 53 churners; Gradient Boosting and Histogram Gradient Boosting reach recall 0.283 and 0.189 because they take no class weight. Random Forest is version-sensitive on this small split: F1 0.472 here and 0.481 under scikit-learn 1.5.0. The retention rules are heuristics whose business effect was not measured, the explanations are model attributions and not causes, and there was no user study.

V. CONCLUSION

ChurnIQ delivers the path from a customer table to explained, banded customers and retention actions, with 44 unit tests and a 63-check API test passing. Against the project objectives, the three required classifiers were trained and compared on five metrics, imbalance was handled and measured, the main drivers were identified and at-risk customers are presented through a dashboard. On synthetic data Logistic Regression and Random Forest are statistically indistinguishable (F1 about 0.46 to 0.51, ROC-AUC about 0.70 to 0.72), accuracy is a poor guide under imbalance, and the scores are rankings and not probabilities. Limits: synthetic data, a 200-row test set, uncalibrated scores, no SHAP, unmeasured business effect, a single shared demo login.

Objectives met: data acquisition and exploration (synthetic data), cleaning and imbalance handling, comparison of the three classifiers, evaluation on five metrics, identification of the main drivers, and a retention dashboard. Not met, because no real data existed: any claim about accuracy on real customers. The key finding is that the leading models are not separable on this data and that imbalance handling, not model choice, changes recall most (0.189 to 0.660 for Logistic Regression).

Future work: validate on real operator data; recalibrate the scores (Platt or isotonic) and tune the decision threshold to offer costs; add SHAP and the base paper's perturbation explainer; evaluate the retention rules in a controlled campaign; and add accounts, a database and drift tests for production use.

REFERENCES

[1] S.-V. Oprea and A. Bara, "Explaining customer churn through unified perturbation-based XAI soft computing," IEEE Access, vol. 14, pp. 24460-24478, 2026, doi: 10.1109/ACCESS.2026.3663648.

[2] A. K. Ahmad, A. Jafar and K. Aljoumaa, "Customer churn prediction in telecom using machine learning in big data platform," J. Big Data, vol. 6, no. 1, Mar. 2019, doi: 10.1186/s40537-019-0191-6.

[3] T. Vafeiadis, K. I. Diamantaras, G. Sarigiannidis and K. Ch. Chatzisavvas, "A comparison of machine learning techniques for customer churn prediction," Simul. Model. Pract. Theory, vol. 55, 2015.

[4] A. Sikri, R. Jameel, S. M. Idrees and H. Kaur, "Enhancing customer retention in telecom industry with machine learning driven churn prediction," Sci. Rep., vol. 14, no. 1, 2024, doi: 10.1038/s41598-024-63750-0.

[5] A. El Attar and M. El-Hajj, "Explainable AI-driven customer churn prediction: a multi-model ensemble approach with SHAP-based feature analysis," Front. Artif. Intell., vol. 9, Feb. 2026, doi: 10.3389/frai.2026.1748799.

[6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.

[7] IBM, "Telco customer churn," sample data set (7,043 customers, 21 columns), distributed through Kaggle; also used in [1] and [3].

[8] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263-1284, Sep. 2009.

[9] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[11] S. Ramirez, "FastAPI documentation," https://fastapi.tiangolo.com (framework used for the API layer).

[12] H. A. Rasool, K. K. Aljawaheri and A. A. M. Karram, "Explainable customer churn prediction in telecom using ensemble learning and SHAP analysis," Math. Model. Eng. Probl., pp. 4373-4386, Dec. 2025, doi: 10.18280/mmep.121226.

[13] M. Szelag and R. Slowinski, "Explaining and predicting customer churn by monotonic rules induced from ordinal data," Eur. J. Oper. Res., vol. 317, no. 2, pp. 414-424, Sep. 2024, doi: 10.1016/j.ejor.2023.09.028.

[14] D. Y. C. Wang, L. A. Jordanger and J. C.-W. Lin, "Explainability of leverage points exploration for customer churn prediction," in Proc. IEEE Int. Conf. Big Data (BigData), Dec. 2023, pp. 5997-6004, doi: 10.1109/BigData59044.2023.10386857.

[15] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Mach. Intell., vol. 2, no. 1, pp. 56-67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.

[16] M. T. Ribeiro, S. Singh and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2016, pp. 1135-1144, doi: 10.1145/2939672.2939778.

[17] D. Slack, S. Hilgard, E. Jia, S. Singh and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in Proc. AAAI/ACM Conf. AI, Ethics, Soc., 2020, doi: 10.1145/3375627.3375830.

[18] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, Oct. 2001, doi: 10.1023/A:1010933404324.

[19] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, 2001.